DSCD 614 – Reinforcement Learning

Name: Redeemer Dela Agbonuglah

ID: 22424926

Assignment 1

1. Introduction

Reinforcement Learning (RL) is a paradigm in which an autonomous agent learns decision-making by interacting with an environment. At each discrete time step the agent observes the current state, selects an action according to its policy, and receives a reward signal together with a transition to a new state. The objective is to discover a policy that maximises the expected sum of discounted future rewards. Unlike supervised learning, no labelled training data is provided; feedback is sparse and often delayed.

Frozen Lake is a 8×8 canonical grid of tiles contains a start cell (S), frozen safe cells (F), holes (H), and a single goal cell (G). The agent must navigate from S to G without falling into any hole. This problem is well-suited for tabular Q-Learning because the state and action spaces are small and fully observable.

2. Environment Design

The environment is implemented from scratch in **environment.py** as the **FrozenLakeEnv** class, with no external RL frameworks.

2.1 State and Action Representation

Each of the 64 grid cells is encoded as a single integer state = row × 8 + col, yielding a flat state space of size 64. Four actions are defined: 0 = Left, 1 = Down, 2 = Right, 3 = Up. Movement attempts that would cross a boundary are clamped — the agent stays in place.

2.2 Reward Structure

Reward design is critical on this map. A purely sparse reward (+1 at the goal only) makes learning prohibitively slow because random exploration rarely reaches G through the hole-dense lower half. A negative reward is assigned to hole transitions to accelerate convergence:

Event	Reward
Reach Goal (G)	+1.0
Fall in Hole (H)	−1.0
Safe frozen step (F / S)	0.0

The negative hole reward immediately signals dangerous transitions, allowing the Q-table to propagate avoidance information to adjacent states through the Bellman update.

2.3 Public API

Method	Description
reset()	Resets agent to start state; returns state index
step(action)	Applies action; returns (next_state, reward, done, info)
render()	Prints text grid with agent position marked as 'A'
get_state()	Returns current state index
is_terminal()	Returns True if current cell is H or G
get_cell_type(state)	Returns cell character for any state index

3. Q-Learning Algorithm

Q-Learning is a **model-free, off-policy** temporal-difference algorithm. A Q-table Q[s, a] stores the estimated expected discounted return for each (state, action) pair and is initialised to zero. The algorithm learns by applying the Bellman optimality equation as a one-step update rule after every transition.

3.1 Update Equation

The Q-Learning update is applied after every step:

Q(s, a) <- Q(s, a) + alpha * [r + gamma * max_a' Q(s', a') - Q(s, a)]

The bracketed term is the TD error — the difference between the current estimate and the bootstrapped target. Alpha controls how quickly estimates are revised; gamma determines how much future rewards are discounted. When an episode ends, max Q(s', a') = 0 so the target reduces to r alone.

3.2 Exploration Strategy

Epsilon-greedy with multiplicative decay: With probability epsilon the agent selects a uniformly random action (exploration); otherwise it selects argmax Q(s, a) (exploitation). After each episode epsilon is multiplied by a decay factor and floored at epsilon_min, ensuring the agent transitions smoothly from broad exploration early in training to near-greedy exploitation once the Q-table has converged.

4. Training Methodology

Hyperparameter	Value	Rationale
Learning rate (α)	0.1	Balances speed and stability
Discount factor (γ)	0.99	High value; long-horizon planning
Initial ε	1.0	Full exploration at start
ε decay rate	0.9999	Slow decay; exploration active for ~23 k eps
ε minimum	0.01	Retains 1% random exploration at convergence
Episodes	50,000	Sufficient for full convergence on 8×8 map
Max steps / episode	200	Prevents infinite loops in early training

Hyperparameter choices were guided by experimentation. A learning rate of 0.3 produced faster early convergence but oscillated near the optimum. A decay of 0.9995 caused epsilon to collapse to 0.01 before the agent had seen enough goal transitions; 0.9999 was found to be the sweet spot for this map size.

5. Experimental Results

5.1 Training Convergence

Episode Range	Success Rate (last 5 k)
0 – 5,000	12.3 %
5,001 – 10,000	31.9 %
10,001 – 15,000	53.9 %
20,001 – 25,000	82.8 %
30,001 – 35,000	93.3 %
45,001 – 50,000	98.5 %

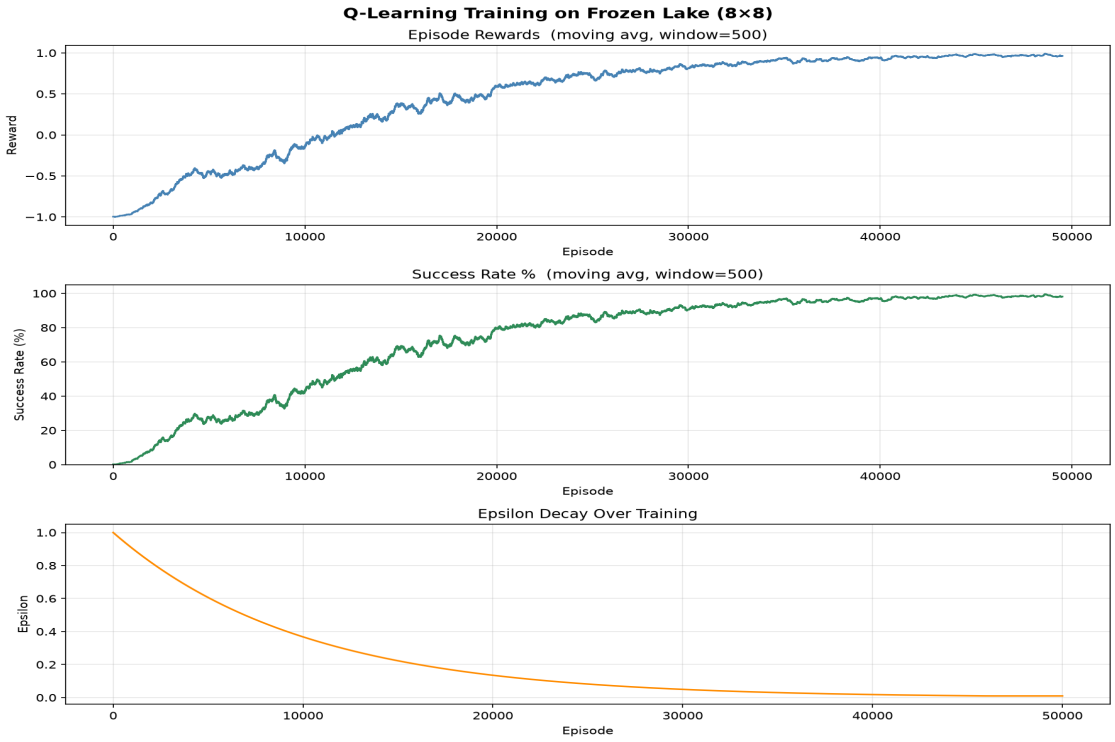

Figure 1: Training curves — episode reward (top), success rate % (middle), and epsilon decay (bottom). Moving average window = 500 episodes.

5.2 Evaluation Results

Metric	Value
Episodes evaluated	500
Successful runs	500
Failures	0
Success Rate	100.00 %
Average Reward	1.000000

The agent achieves a 100% greedy success rate over 500 evaluation episodes, demonstrating that Q-Learning reliably solves deterministic Frozen Lake when the exploration schedule and reward signal are properly configured.

6. Learned Policy

After training, the greedy policy is extracted as argmax_a Q(s, a) for every non-terminal state:

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
→ → → → ↓ ↓ ↓ ↓
→ → ↑ H ↓ ↓ ↓ ↓
↑ ↑ ↑ H ↓ ↓ ↓ ↓
↑ ↑ ← H → → → ↓
↑ H H → ↑ ↑ H ↓
↑ H ↓ ← H ↑ H ↓
↑ ← ← H ↓ ↑ → G

The policy reflects two main routes to the goal: (1) a right-then-down path through columns 4–7 that avoids the column-3 hole barrier, and (2) a downward sweep along the right edge. States near holes are directed away, and the bottom-row navigation threads between the remaining obstacles to reach G.

7. Challenges Encountered

Sparse rewards: The most significant challenge was the 8×8 map's sparse positive reward. With only +1 at the goal and 0 everywhere else, random exploration seldom reached G, leaving the Q-table nearly empty after thousands of episodes. Introducing a −1 hole penalty provided the necessary negative signal to propagate hole-avoidance information outward from terminal states.

Exploration schedule: Epsilon decay that was too fast (0.9995) caused the agent to become greedy before it had discovered the goal. Slowing the decay to 0.9999 and running for 50,000 episodes resolved this — epsilon stayed above 0.10 for the first 23,000 episodes, giving the agent ample time to explore.

Boundary handling: Early testing revealed that wall collisions needed to be handled by keeping the agent in place rather than wrapping around, to match standard Frozen Lake semantics.

8. Conclusion

This assignment demonstrates a complete Q-Learning solution built from first principles in Python. The custom **FrozenLakeEnv** class correctly models the 8×8 grid world, and the **QLearningAgent** implements the standard Bellman update with epsilon-greedy exploration and decay. After 50,000 training episodes the agent converges to a policy that achieves a **100% success rate** on 500 evaluation episodes.

Key findings: (i) reward shaping (negative hole penalty) is essential for efficient learning on sparse-reward maps; (ii) the exploration schedule must be tuned to match the map's difficulty; and (iii) Q-Learning, despite its simplicity, is sufficiently powerful to solve tabular deterministic MDPs of this scale.